

A Comprehensive Study for Software Testing and Test Cases Generation Paradigms

Roaa Elghondakly

Department of Information
Systems, Faculty of Computer and
Information Sciences,
Ain Shams University, Cairo
11566, Egypt
roaa.ahmed@cis.asu.edu.eg

Sherin Moussa

Department of Information
Systems, Faculty of Computer and
Information Sciences,
Ain Shams University, Cairo
11566, Egypt
sherinmoussa@cis.asu.edu.eg

Nagwa Badr

Department of Information
Systems, Faculty of Computer and
Information Sciences,
Ain Shams University, Cairo
11566, Egypt
nagwabadr@cis.asu.edu.eg

ABSTRACT

Software testing is accounted to be an essential part in software development life cycle in terms of cost and manpower, where its total cost is considerable high. Consequently, many studies [48] have been conducted to minimize the associated cost and human effort to fix bugs and errors, and to improve the quality of testing process by automatically generating test cases. Test cases can be generated from different phases (requirement phase, design phase and after development phase). Though, test case generation at early stages is more effective rather than that after development, where time and effort used for finding and fixing errors and bugs is less than that after development. At later stage, fixing errors results in enormous code correction, consuming a lot of time and effort. In this paper, we study the different paradigms of testing techniques for generating test cases, where we investigate their coverage and associated capabilities. We finally propose a preliminary model for a generic automated test cases generation.

Keywords

Software Testing; Code-based Testing; Model-based Testing; Search-based Testing; Requirements-based Testing and Test Case Generation.

1. INTRODUCTION

Software testing is considered one of important phases in software development life cycle. The development life cycle total cost is considerable to be high. For this reason, it is important to minimize this cost and the human effort to fix bugs and errors, as well as to improve the quality of the testing process by automating it. Test automation [27] is the process of using separate software to manage and evaluate the fulfillment of test cases and to compare the expected outcomes with the generated ones. Moreover, the automation process is used to generate test case with minimum human intervention, which will consequently help to minimize total cost. Testing process is considered the key to success of any software. Testing may be performed in the last phase of the software development life cycle or at an early stage.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.

ICC '16, March 22-23, 2016, Cambridge, United Kingdom
© 2016 ACM. ISBN 978-1-4503-4063-2/16/03...\$15.00
DOI: <http://dx.doi.org/10.1145/2896387.2896435>

In case of test automation, if the process is carried out at early phase of the development life cycle, it saves more time and effort to detect errors. At later stages, enormous errors would be generated as soon as the code is completed, where it demands a lot of code correction and modification. Different studies are carried out for test case generation techniques [1].

In this paper, we present a crucial study that covers different test case generation techniques. In addition, a comparison between different test case generation types is provided. "Figure 1" shows the survey architecture. The rest of the paper is organized as follows: Section 2 discusses the code-based testing techniques. Section 3 illustrates other different testing techniques that are model-based. Section 4 investigates search-based testing techniques. Section 5 covers requirement-based testing from different models i.e.: waterfall model and agile model. Finally, Section 6 summarizes the demonstrated studies and presents our analysis.

2. CODE-BASED TESTING

CODE-BASED testing techniques are carried out on code. In case of testing in the last phase of software development life cycle, it will not be effective as in early stage, since enormous errors will be generated as soon as the code is completed. This demands a lot of code correction and modification. Thus, the testing process should be carried out at the beginning of software development life cycle (i.e. requirements and design phases) in order to save time and cost. In [78], the authors used immune operators to support a test cases generation algorithm based on immune principles. In this work, authors depended on code development in test case generation process, which led to detecting errors lately and badly affected time and cost.

Dahiya et al [12] proposed a particle swarm optimization (PSO) based hybrid testing technique named as "pseudo dynamic testing" for test data generation from C programs by accomplishing demanding test criteria: the all-path testing criterion using an interpreter. This methodology tried to solve many of the structural testing problems such as input dependent array index, dynamic variables, in-feasible paths, abstract function calls and loop handling. The technique was applied on real world programs. The set of test inputs generated were not redundant as each leads to a different program path. They depended totally on source code in the process of test case generation, which led to the same problems as stated before. Code-based testing is not preferable in the process of test case generation, as it must be carried out lately after the implementation (development) phase. This leads to enormous errors during testing that will consume a lot of time and cost to be solved rather than that at early stages.

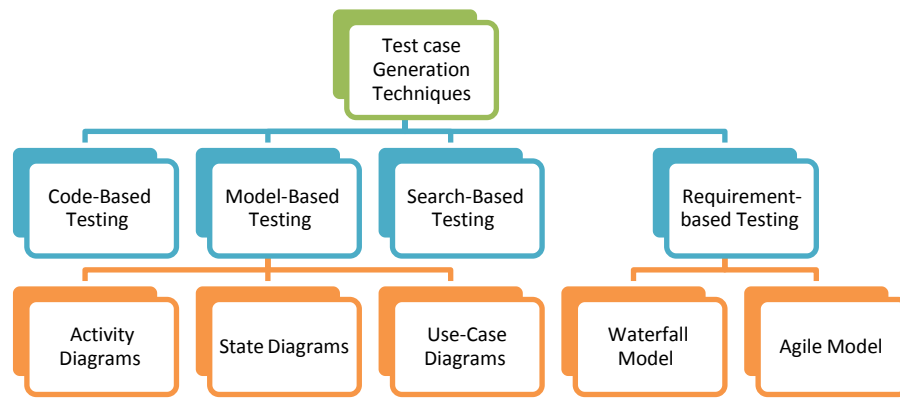


Figure 1. Survey Architecture.

3. MODEL-BASED TESTING

Due to the problems encountered in code-based testing, hence the testing process must be carried out at early stage (i.e. requirement and design) phases. For this reason, model-based testing [50], [24] was preferable rather than code-based testing [78], [12] to generate test cases from Unified Modeling Language (UML) diagrams (i.e. behavioral diagrams) during the design phase.

UML [4] is divided into two main parts; structural diagrams (i.e. class diagram) and behavioral diagrams (i.e. activity, use-case and state diagrams). Structural diagrams are used to show the structure, style or design of the software while behavioral ones are used to clarify the steps in which the software will pass through until it reaches the desired output. In other words, it shows the flow of events.

As model-based software development becomes more important, the relevance of model-based test cases generation increases as well. The ability to cover multiple UML models is appealing. In the last decade, test case generation became an important research topic. Many researchers use behavioral diagrams (i.e. use case, activity and state) in order to generate test cases. However, most of the previous studies used different techniques but they encounter some limitations, as not being generic [25], [6], [33], [9], [8] in which they are limited to only one type of diagrams in each single method.

In addition, some were presenting manual approaches [41], [35], [49], [70], [18], [26], which are not time saving. It is possible to build automatic tools based on these approaches, which are expected to reduce cost of software development and improve software quality. Whereas the alternating variable method used in [61] to generate test data does not provide globally optimal solution. As well as in [71], it does not provide optimization. Moreover, developing complicated methods to generate test cases is considered a limitation as in [62], [17]. In the following subsequent sections, we present the different related studies categorized according to the type of the behavioral diagrams.

3.1 Activity Diagram

The studies in [72], [46], [47], [49] used the activity diagram to generate test cases. A dependency table for the diagram was generated as well as a corresponding graph. The graph was then parsed to find all possible paths between the start and end nodes. Authors of [7], [35] stored the activity diagram as XML Metadata Interchange (XMI) files, and then retrieved the required information from the XMI file. Mapping rules were then applied to generate a corresponding graph from which test paths and test

cases were generated respectively. Li et al [37] built an algorithm to generate test sequences by applying Euler circuit algorithm to decrease the number of generated test cases. On the other hand, Ray et al [55] reduced the number of generated test cases by applying conditioned slicing on a predicate node of the graph generated from the activity diagram. Whereas Sumalatha et al [69] integrated the activity diagram with the sequence diagram. They used the corresponding graphs generated from both diagrams to generate test cases.

3.2 State Diagram

As for the state diagrams, a technique was presented in [32], where a state diagram was transformed to flow graph to help in the test cases generation process. Genetic algorithms were used as well in the test case generation process as in [68], [14].

3.3 Use-Case Diagram

Concerning use-case diagrams, the use-case diagram was transformed to activity diagram in [18], where test cases were generated by the different methods as stated earlier in activity diagrams subsection. Different tools and methods in [76] were used to convert use-case diagram to activity diagram [29], [30], [60], [74], [22], [21]. Javier J. Gutiérrez et al. in [30] proposed an approach where use-case diagram was transferred to activity with QVT-Relational. Use-cases were described as activity charts. Whereas, authors in [60] dealt with use-case models as textual use case descriptions and converted it to activity diagrams where each use case has its corresponding activity diagram.

In [74], an approach was proposed for generating activity diagrams from use cases automatically while establishing traceability links, as well as data flow information can be generated and added to these activity diagrams. Javier J. Gutiérrez, et al. in [22] converted use-case diagrams to activity diagrams, where the use case is coded as a XML document. Each use case definition is automatically translated into one activity diagram. To perform this process, a set of pattern language and translation algorithms is needed. The pattern language allowed the automatically extraction of the relevant information from the steps of a use case so that the algorithms can build the activity diagram from that information.

In [21], they used the requirement model proposed by the Navigational Development Technique (NDT) [16]. Then a behavioral model is built from the use case, and finally test cases were generated. Despite of the importance of all UML diagrams and the enrichment they provide to better describe systems, most of the researches generated test cases only either from activity

diagrams, sequence diagrams or both [46], [47], [66], [67], [3]. The approaches in [72], [69] combined activity and sequence diagrams together to generate test cases.

4. SEARCH-BASED TESTING

Search Based Software Testing (SBST) [15],[28], [39], [38] is a branch of Search Based Software Engineering (SBSE), in which optimization algorithms are used to allow dynamic search for test data to maximize the achievement of test goals and minimize testing costs. As a reduced set of test cases is generated and executed, the total testing time will be minimized as well as the total cost. In fact, the optimization process has different techniques as reducing the number of test cases, test cases prioritization, as well as minimizing time, increasing performance, and maximizing the quality of outcomes, etc.

Several researches [15], [38] have studied some algorithms as Hill Climbing, Simulated Annealing and Genetic algorithms. The first two algorithms were used with the early researches, but they were used with reference to only one elected solution at one time. For this reason, using genetic algorithms (GAs) in [14] became more efficient as they were global searches, and can sample more than one point at the same time in the search space. Although GAs have many advantages, they cannot solve specific optimization problems. This happens because fitness functions are not known. Besides, there is no assurance that they will find a global optimum. In addition, genetic algorithm real-time applications are limited due to the random solutions and convergence.

[2] Has proved that Ant colony optimization (ACO) [36], [13] is better than GAs, as it overcomes some of the disadvantages found by GAs, i.e. there is no assurance that GAs will find a global optimum. Besides, specific optimization problems cannot be solved because the fitness functions are not known. In addition, ACO is better than other global optimization techniques for TSP as neural network techniques, and simulated annealing. In addition, it reserves an entire colony memory instead of the previous generation only. Moreover, the combination of colony memory and the random path selection makes it less affected by poor initial solutions. ACO can be used in dynamic applications, where it can be adjusted to real-time applications' changes as it runs continuously.

5. REQUIREMENT-BASED TESTING

Requirements-based testing is a testing approach in which test cases are derived from requirements. Requirements represent the initial phase in software developments life cycle. Any ambiguity in natural language requirements may result in magnificent errors in the next phases. In addition, poorly defined requirements may cause software project failure. There exist many software development models as waterfall model, agile model, etc. Requirements can be gathered from different models either waterfall model (functional and non-functional) or agile model' user stories.

Requirements-based testing is defines as it is a testing approach that generates test cases from requirements. Despite it is not easy to perform complete testing from requirements as not all requirements can be tested by single test cases, while others need set of test cases to verify. Moreover, even when a full set of requirements-based test cases are applied to a system, this does not ensure that the entire system has been tested [31]. In order to increase the quality of result it is more effective to test requirements at early stages. In waterfall models, testing can be performed after development and implementation phase where

executable software (even if partially complete) exists. In contrast, under an agile approach, requirements, programming, and testing are predominating take place concurrently. According to the continuous increasing and changing of requirements, customer involvement is mandatory. Agile development is aligned with the view to face the challenges of an increasingly volatile marketplace, changing requirements, customer involvement, priorities and shorter deadlines [59].

On the other hand in case of functional requirements, it is too complicated to perform further processes on them, as generating a test case process, where functional requirements are mostly written using use cases with textual description. In software requirement, whether functional or non-functional, it identifies a system capability, quality, necessary attributes and characteristics as well as it describes a utility to an internal user, a customer, or other stakeholder. Requirements are mostly written using use cases model (i.e. a textual description for every use case flow of events). This textual description can be adopted easily. However, it is too complicated to perform further processes on them, such as generating test cases.

Many studies have used textual requirements to build state-charts either manually or automatically [45]. Some approaches translated natural language requirements automatically to a logical or systematic format to be able to generate test cases from textual requirements. Others neglected textual requirements and replaced them with a use case diagram, which is then transformed to an activity diagram [29]. In addition, some approaches converted natural language requirements to a flow graph (diagram) then extracted test paths from it by applying path coverage methods [40].

Other approaches performed some coverage criteria that combined Boolean propositions [57] on natural language requirements. Most of the done work handled requirements as a use case model then converted it to an activity or state diagram to ease the process of test case generation, where it is claimed that requirements must be translated to a formal model to generate test cases from requirements.

Neringa Sipavičienė et al. in [43] used use case diagrams in addition to activity diagrams to specify requirements and generate test cases. In [58], the authors stated that natural language functional requirements are not applicable or valid to generate test cases. Tao Yue, Lionel et al. in [75] performed a systematic review to survey the existing approaches that used models to deal with requirements. This review, as well as [31], confirmed that most of the related work studies preferred translating requirements into an analysis model for further usage.

The authors of [53] presented a tool suite called RETNA that generated test cases from natural language requirements based on state machines. On the other hand, Priyanka Kulkarni et al. in [52] proposed a method that applied natural language processing techniques on requirements, where they constructed a graph using Hadoop, and then they traversed it to generate test cases. Whereas in [64], Saurabh Tiwari et al. proposed another approach using agile models to generate test cases from requirements, where use case specifications were transformed into an activity diagram per actor.

6. PROPOSED APPROACH

Based on our extensive studies, we propose an approach in which model-based testing is combined with search-based testing to formulate a dynamic, generic, automated and

optimized model with minimum human intervention (Phase I). The proposed model is expected to be generic enough to generate test cases from any UML diagram regardless its type (use-case, activity, state...etc.) for multi-disciplinary domains.

For each diagram, an enhanced weighted graph is constructed, where an EACO (Enhanced Ant Colony Optimization) technique is proposed that selects the next node modified using the enhanced graph. The graphs generated from UML behavioral diagrams are un-weighted. Thus, the proposed model employs weighted graphs, where weights are added to the edges. These weights are calculated to be the maximum number of children between any two nodes forming an edge. This number is used to ensure that each node has at least one child, which guarantees the continuous connection between nodes. Depth First Search (DFS) is then used to parse the enhanced graph and find all test paths between initial and final nodes. DFS is used as it uses less memory in comparison to that used by Breadth First Search (BFS), as it is not necessary for each level to store all the child pointers.

EACO (Enhanced Ant Colony Optimization) algorithm is applied on the generated set of test cases to find the optimum test case with the shortest path and highest pheromone value. A test data generation technique [23], [76], [65], [54], [73], [34], [10], [77] is used to validate the optimum test case. The proposed model can be used to generate test cases automatically from different behavioral diagrams, as it can be used for state, activity, use-case diagrams. In addition, it meets coverage criteria (statement coverage, branches coverage and paths coverage). Not only optimized test cases are generated, but also a test data validation technique is applied to ensure the correctness of the optimum generated test cases. The proposed model is optimized, where it reduces the execution time in comparison to that used by DFS and BFS. It meets the cyclomatic complexity [11] values when applied to some of the related work examples.

Moreover, another phase (Phase II) was proposed to automatically generate test cases from requirements in addition to requirements validation [56]. The proposed approach is applied on various software models (i.e. waterfall and agile models), as well as multi-disciplinary domains. It also applicable for different requirements forms as SRS documents (i.e. functional requirements, non-functional requirements, and use case models) also it is applied on to user stories in agile models. The proposed approach generates optimized test cases where it uses text mining in addition to symbolic execution methodology for test data generation and for validating the generated test cases, where a knowledge base is developed for multi-disciplinary domains. Efficiency is proved, where the required effort for generating a test case decreases after optimization due to increasing the number of generated test cases.

"Table 1" represents a comparison between the proposed approach (Phase I and Phase II) and the different model-based and requirement-based testing studies. The comparison is based on several criteria, including whether the related approaches are

automated or not, generate test paths or do not, generate test cases... etc.

Analyzing "Table 1", it is found that most of the related work studies dealt with only one diagram or two at most in each single approach. In addition, only one or two studies apply test data generation techniques and perform optimization. While in case of requirement-based testing studies, where most of the related work studies handle waterfall models only. In addition, some use natural language requirements only to generate test cases. Whereas others combine UML specification with natural language requirements, as they believe that UML diagrams are specification for requirements that can be used later in further process as test case generation.

7. DISCUSSION

Considering the previous studies, it was found that model-based testing as well as requirement-based testing techniques are preferable rather than code-based testing techniques, as they can detect errors at early stages of requirements and design, which allows saving time and cost. In addition, search-based testing techniques improve software quality by applying optimization techniques. Thus, combining model-based testing with search-based testing through an optimization procedure in our first phase of the proposed approach significantly enhances the test case generation process. Whereas in the second phase, we applied requirement-based testing techniques in which test case generation process can be carried out from different requirements types as well as different software development models (Waterfall and Agile). Test data generation and validation techniques are then performed, which dramatically minimize human effort and cost of test case generation process.

8. CONCLUSION AND FUTURE WORK

Software testing is an essential part in software development life cycle. The associated total cost is accounted to be high, especially when carried at later stages after development phase, in which a lot of effort and time is needed as well as major code correction.

In this paper, we provided a comprehensive study for the different testing paradigms used as code-based testing, model-based testing, requirements-based testing and search-based testing. We investigated each type and discussed its variable approaches. Based on our studies, requirement-based testing as well as model-based testing are more efficient for the test case generation process, in which test cases are generated at early stage before development phase.

On the other hand, code-based testing consumes a lot of time and effort as it depends on the generated code, leading to major code correction and modification. In addition, we propose a preliminary model for a generic automated test cases generation. In future, the proposed approach can be studied to be applied on regression testing. In addition, additional UML diagrams can be considered for conversion, such as the sequence diagram, class diagram ... etc.

TABLE 1. COMPARISON BETWEEN PROPOSED APPROACH (PHASE I AND PHASE II) AND DIFFERENT MODEL-BASED AND REQUIREMENT-BASED TESTING STUDIES

References name/no.	UML specification	Waterfall / Agile model	Natural Language requirements (NLR)/ User story	Automated	Generate Test Paths	Generate Test cases	Generate Reduced Set of Test Paths	Generate Reduced Set of Test Cases	Generate Optimum Test Path	Generate Optimum Test case	Apply Test Data Generation Technique	Generate Optimum Test Case Validation
Proposed approach (Phase I)	Activity, State, Use-Case	Waterfall	-	✓	✓	✓	✓	✓	✓	✓	✓	✓
Proposed approach (Phase II)	-	Waterfall, Agile	(NLR), User Story	✓	✓	✓	✓	✓	✓	✓	✓	✓
[72]	Activity with Sequence	Waterfall	-	✓	-	✓	-	-	-	-	-	-
[25]	Activity	Waterfall	-	✓	✓	-	-	-	-	-	-	-
[46]	Activity	Waterfall	-	✓	✓	✓	-	-	-	-	-	✓
[47]	Activity	Waterfall	-	✓	✓	✓	✓	✓	-	-	-	✓
[41]	Activity	Waterfall	-	✓	✓	✓	-	-	-	-	-	-
[7]	Activity	Waterfall	-	✓	✓	✓	-	-	-	-	-	-
[35]	Activity	Waterfall	-	✓	✓	✓	-	-	-	-	-	-
[6]	Activity	Waterfall	-	✓	✓	✓	-	-	-	-	-	-
[37]	Activity	Waterfall	-	✓	✓	✓	✓	✓	-	-	-	-
[49]	Activity	Waterfall	-	-	✓	-	-	-	-	-	-	-
[55]	Activity	Waterfall	-	✓	✓	✓	-	-	-	-	-	-
[70]	Activity	Waterfall	-	✓	✓	✓	✓	✓	-	-	-	-
[33]	Activity	Waterfall	-	-	✓	-	-	-	-	-	-	-
[69]	Activity, Sequence	Waterfall	-	✓	✓	✓	-	-	-	-	-	-
[32]	State	Waterfall	-	✓	✓	✓	-	-	-	-	-	-
[61]	State	Waterfall	-	✓	✓	✓	✓	✓	-	-	✓	-
[68]	State	Waterfall	-	✓	✓	✓	-	-	-	-	-	-
[62]	State chart, Finite State Behavioral Models	Waterfall	-	✓	✓	✓	-	-	-	-	-	-
[17]	State	Waterfall	-	✓	✓	✓	-	-	-	-	-	-
[71]	State	Waterfall	-	✓	✓	✓	-	-	-	-	-	-
[9]	State	Waterfall	-	✓	✓	✓	✓	✓	-	-	-	-
[8]	State	Waterfall	-	✓	✓	✓	-	-	-	-	-	-
[36]	State	Waterfall	-	✓	-	-	-	-	✓	-	-	-
[14]	State	Waterfall	-	✓	✓	✓	-	-	-	-	✓	-
[5]	Use-Case	Waterfall	-	✓	✓	✓	-	-	-	-	-	-
[42]	Use-Case	Waterfall	-	✓	✓	✓	-	-	-	-	-	-
[18]	Use-Case	Waterfall	-	✓	✓	✓	-	-	-	-	-	-
[19]	Use-Case	Waterfall	-	✓	✓	✓	-	-	-	-	-	-
[26]	Use-Case	Waterfall	-	-	✓	✓	-	-	-	-	✓	-
[63]	Use-Case, Sequence	Waterfall	-	✓	-	✓	-	-	-	-	-	-
[20]	Use-Case	Waterfall	-	✓	✓	✓	-	-	-	-	-	-
[43]	Use-Case	Waterfall	-	✓	-	-	-	-	-	-	-	-
[58]	State-Chart	Waterfall	(NLR)	✓	-	-	-	-	-	-	-	-
[75]	Different diagrams	Waterfall	(NLR)	✓	-	-	-	-	-	-	-	-
[53]	-	Waterfall	(NLR)	✓	-	-	-	-	-	-	-	-

[29]	Use-Case, Activity	Waterfall	-	✓	-	-	-	-	-	-	-	-
[52]	-	Waterfall	(NLR)	✓	-	-	-	-	-	-	-	-
[31]	-	Waterfall	(NLR)	✓	-	-	-	-	-	-	-	-
[64]	Use-Case, Activity	Agile	-	-	-	-	-	-	-	-	-	-

9. REFERENCES

- [1] Anand, Saswat, Edmund K. Burke, Tsong Yueh Chen, John Clark, Myra B. Cohen, Wolfgang Grieskamp, Mark Harman, Mary Jean Harrold, and Phil McMinn, 2013. An orchestrated survey of methodologies for automated software test case generation, *Journal of Systems and Software* 86, no. 8: 1978-2001, 2013.
- [2] Ant_Colony_Optimization_powerpoint_ppt_presentation, <http://www.powershow.com/view/3d52cb-YzUzO/>
- [3] Bao-Lin Lin, Zhi-shu Li, 2007. Test Case automate Generation from UML Sequence diagram and OCL Expression. *International Conference on Computational Intelligence and Security*.
- [4] Bipin Pandey, Rituraj Jain, 2014. Importance of Unified Modeling Language for Test Case Generation in Software Testing. *Int.J.Computer Technology & Applications*, Vol 5 (2), 345-350. IJCTA.
- [5] Chen Lizhe, Li Qiang, 2010. Automated Test Case Generation from Use Case: A Model Based Approach. IEEE.
- [6] Chen Mingsong, Mishra Prabhat, Kalita Dhrubajyoti, 2008. Coverage-driven Automatic Test Generation for UML Activity Diagrams. ACM.
- [7] Chen Mingsong, Mishra Prabhat, Kalita Dhrubajyoti, 2010. Efficient test case generation for validation of UML Activity diagrams. Springer Science+Business Media.
- [8] Chimisliu Valentin, Wotaw Franz, 2012. Model Based Test Case Generation for Distributed Embedded Systems. IEEE.
- [9] Chimisliu Valentin, Wotawa Franz, 2013. Improving Test Case Generation from UML Statecharts by using Control, Data and Communication Dependences. *International Conference on Quality Software*.
- [10] Clarke Lori, 1976. A System to Generate Test Data and Symbolically Execute Programs. IEEE.
- [11] Cyclomatic Complexity. <https://sites.google.com/site/swtesting/concepts/home/test-design-techniques/for-white-box/statement-branch-and-path-coverage>.
- [12] Dahiya S. S., Chhabra J. K., 2011. Pso based pseudo dynamic method for automated test case generation using interpreter. Springer-Verlag Berlin Heidelberg.
- [13] Dorigo M., Birattari M., 2006. Ant colony optimization artificial ants as a computational intelligence technique. IEEE.
- [14] Dounsa-ard, Chartchai, Dahal, Keshav et al, 2007. An Automatic Test Data Generation from UML State Diagram using Genetic Algorithm. EAST-WEST project.
- [15] Enou E. P., Doganay K., 2013. Mos: An integrated model-based and search-based testing tool for function block diagrams. CMSBSE.
- [16] Escalona M.J. 2004. Models and Techniques for the Specification and Analysis of Navigation in Software Systems. Ph. European Thesis. Department of Computer Language and Systems. University of Seville. Seville, Spain.
- [17] Gnesi Stefania, Latella Diego et al, 2001. Formal Test-case Generation for UML Statecharts. EU-IST IST.
- [18] Gutiérrez J.J., Escalona María J. et al, 2006. An Approach to Generate Test Cases from Use Cases. ACM.
- [19] Gutiérrez J.J., Escalona María J. et al, 2007. Derivation of test objectives automatically. Springer-Verlag Berlin Heidelberg.
- [20] Gutiérrez J.J., Escalona María J. et al, 2007. Using Use Case scenarios and Operational Variables for Generating Test Objectives. The Ministry of Science and Education (Spain).
- [21] Gutiérrez, J. J., Escalona, M. J., Mejías, M., & Torres, J., 2007. Derivation of test objectives automatically. In *Advances in Information Systems Development* (pp. 435-446). Springer US.
- [22] Gutiérrez, J. J., Escalona, M. J., Mejías, M., & Zenteno, A. H., 2007. Using Use Case scenarios and Operational Variables for Generating Test Objectives. *Systems Testing and Validation*.
- [23] Harman Mark, Jia Yue, 2011. Strong Higher Order Mutation-Based Test Data Generation. ACM.
- [24] Hartmann J., Vieira M., 2004. Uml-based test generation and execution. Siemen Corporate Research.
- [25] Hettab Abdelkamel, Chaoui Allaoua, Aldahoud Ahmad, 2013. Automatic Test Cases Generation from UML Activity Diagrams Using Graph Transformation. *The 6th International Conference on Information Technology*.
- [26] Heumann Jim, 2001. Generating Test Cases from Use Cases. Rational Software.
- [27] https://en.wikipedia.org/wiki/Test_automation
- [28] Iqbal M. Z., Arcuri A., 2012. Combining search-based and adaptive random testing strategies for environment model-based testing of real-time embedded systems. ACM.
- [29] Javier J. Gutiérrez, Clémentine Nebut, María J. Escalona, Manuel Mejías, Isabel M. Ramos, 2008. Visualization of Use Cases through Automatically Generated Activity Diagrams, *Model Driven Engineering Languages and Systems*. Springer Berlin Heidelberg, pp. 83-96, 2008.
- [30] Jesús M. Almendros-Jiménez and Luis Iribarne, 2005. Describing Use Cases with Activity Charts, Springer-Verlag Berlin Heidelberg.
- [31] John Lee and Jon Friedman, 2014. Requirements Modeling and Automated Requirements-Based Test Generation, SAE International.
- [32] Kansomkeat Supaporn, Rivepiboon, 2003. Automated-Generating Test Case Using UML Statechart Diagrams. SAICSIT.
- [33] Kim Hyungchoul, Kang Sungwon et al, 2007. Test Cases Generation from UML Activity Diagrams. *International Conference on Software Engineering, Artificial Intelligence, Networking, and Parallel/Distributed Computing*.
- [34] Korel Bogdan, 1990. Automated Software Test Data Generation. IEEE.
- [35] Kundu Debasish, Samanta Debasish, 2009. A novel approach to generate test cases from UML activity diagrams. *Journal of Object Technology*.
- [36] Li Huaizhong, Lam C. Peng, 2007. Software Test Data Generation using Ant Colony Optimization. *International Journal of Computer, Information Science and Engineering*.
- [37] Li Liping, Li Xingsen et al, 2013. Extenics-based Test Case Generation for UML Activity Diagram. *Information Technology and Quantitative Management*.
- [38] McMinn P, 2004. Search-based software test data generation: a survey. *Software Testing, Verification and Reliability*.
- [39] McMinn P., 2011. Search-based software testing: Past, present and future. IEEE.

- [40] Naresh Ahlowalia, 2002. Testing from use cases using path analysis technique. In *International Conference on Software Testing Analysis and Review*.
- [41] Nayak Ashalatha, Samanta Debasis, 2009. Synthesis of test scenarios using UML activity diagrams. Springer-Verlag Berlin Heidelberg.
- [42] Nebut Cle'mentine, Fleure Franck et al, 2006. Automatic Test Generation: A Use Case Driven Approach. IEEE.
- [43] Neringa Sipavičienė, Kristina Smilgytė, and Rimantas Butleris. Information Systems Requirements Specification and Usage in Test Case Generation, ICIST, Springer International Publishing Switzerland pp. 24-34, 2014.
- [44] Nuri Jazuli Bin Kamarudin, Nor Fazlida Mohd Sani, Rodziah Atan, 2012. Transformation from Requirement into Behavior Design: A Review, IEEE.
- [45] P. Frohlich and J. Link, 2000. Automated test case generation from dynamic models. ECOOP 2000—Object-Oriented Programming. Springer Berlin Heidelberg, pp. 472-491, 2000.
- [46] Pakinam Boghdady, Nagwa Badr, 2011. A Proposed Test Case Generation Technique Based on Activity Diagrams. International Journal of Engineering & Technology IJET-IJENS.
- [47] Pakinam Boghdady, Nagwa Badr, 2011. An Enhanced Test Case Generation Technique Based on Activity Diagrams. IEEE.
- [48] Passant Kandil, Sherin Moussa and Nagwa Badr, 2015. A Study for Regression Testing Techniques and Tools. International Journal of Soft Computing and Software Engineering [JSCSE], 5(4), pp. 64-84, 2015.
- [49] Pechtanum kanjane, kansomkeat supaporn, 2012. Generation test case from UML activity diagram based on ac grammar. In *International Conference on Computer and Information Science*.
- [50] Prasanna M., Chandra K. R., 2011. Automatic test case generation for uml collaboration diagrams. IETE Journal of Research.
- [51] Prasanna M., Chandran K. R., 2011. Automated test case generation for object oriented systems using uml object diagrams. Springer-Verlag Berlin Heidelberg.
- [52] Priyanka Kulkarni and Yashada Joglekar, 2014. Generating and Analyzing Test cases from Software Requirements using NLP and Hadoop, International Journal of Current Engineering and Technology (INPRESSCO).
- [53] R. Boddu, L. Guo, S. Mukhopadhyay, and B. Cukic, 2004. Retna: From requirements to testing in a natural way, In RE04 (Requirements Engineering), pp. 262–271, Japan.
- [54] Ramamoorthy C. V., F. HO SIU-BUN, 1976. On the Automated Generation of Program Test Data. IEEE.
- [55] Ray Mitrabinda, Barpanda Soubhagya, Mohapatra Durga, 2009. Test Case Design Using Conditioned Slicing of Activity Diagram. International Journal of Recent Trends in Engineering.
- [56] Roaa Elghondakly, Sherin Moussa, Nagwa Badr, Waterfall and Agile Requirements-based Model for Automated Test Cases Generation. *The Seventh IEEE International Conference on Intelligent Computing and Information Systems (ICICIS 2015)*.
- [57] Ruder, 2004. Uml-based test generation and execution. Rückblick Meeting.
- [58] Ryser, J., Glinz, M., 1999. A Practical Approach to Validating and Testing Software Systems Using Scenarios Quality, Week Europe QWE'99 in Brussels. Institut für Informatik, Universität Zürich.
- [59] S. W. Ambler, 2009. The agile scaling model (ASM): adapting agile methods for complex environments, Environments, IBM.
- [60] Sadaf Mustafiz, Jörg Kienzle, Hans Vangheluwe, 2009. Model Transformation of Dependability-Focused Requirements Models, IEEE.
- [61] Samuel P., Mall R, bothra A.K, 2008. Automatic test case generation using unified modeling language (UML) state diagrams. The Institution of Engineering and Technology.
- [62] Santiago Valdivino, Vijaykumar N.L. et al, 2008. An environment for automated test case generation from statechart-based and finite state machine-based behavioral models. IEEE.
- [63] Sarma Monalisa, Mall Rajib, 2007. Automatic Test Case Generation from UML Models. *International Conference on Information Technology*.
- [64] Saurabh Tiwari and Atul Gupta, 2015. An Approach of Generating Test Requirements for Agile Software Development, In *Proceedings of the 8th India Software Engineering Conference*, pp. 186-195, ACM.
- [65] Shahbaz Muzammil, McMinn Phil, 2014. Automatic generation of valid and invalid test data for string validation routines using web searches and regular expressions. Science of Computer Programming.
- [66] Shanthi A.V.K., Kumar G. Mohan, 2012. Automated Test Cases Generation from UML Sequence Diagram. *International Conference on Software and Computer Applications*.
- [67] Shirole Mahesh, Kumar Rajeev, 2010. A Hybrid Genetic Algorithm Based Test Case Generation Using Sequence Diagrams. Springer-Verlag Berlin Heidelberg.
- [68] Shirole Mahesh, Suthar Amit et al, 2011. Generation of Improved Test Cases from UML State Diagram Using Genetic Algorithm. ACM.
- [69] Sumalatha V.Mary, Raju G.S.V.P., 2012. UML based Automated Test Case Generation technique using Activity-Sequence diagram. The International Journal of Computer Science & Applications.
- [70] Sumalatha V.Mary, Raju G.S.V.P., 2013. Model based test case generation from uml activity diagrams. The International Journal of Computer Science and Applications.
- [71] Swain Ranjita, Panthi Vikas, 2012. Test case generation based on state machine diagram. International Journal of Computer Information Systems.
- [72] Swain Santosh, Mohapatra Durga, 2010. Test Case Generation from Behavioral UML Models. International Journal of Computer Applications, 0975 – 8887.
- [73] Tahbaldar Hitesh, Kalita Bichitra, 2011. Automated Software Test Data Generation: Direction of Research. Int. Journal of Computer Science & Engineering Survey.
- [74] Tao Yue, Lionel C. Briand, and Yvan Labiche, 2010. An Automated Approach to Transform Use Cases into Activity Diagrams, Springer-Verlag Berlin Heidelberg.
- [75] Tao Yue, Lionel C. Briand, Yvan Labiche, 2011. A systematic review of transformation approaches between user requirements and analysis models, Requirements Engineering 16.2, pp. 75-99, 2011.
- [76] Veanes Margus, Campbell Colin, 2007. Model-Based Testing of Object-Oriented Reactive Systems with Spec Explorer. Springer-Verlag Berlin Heidelberg.
- [77] Xing Ying, Gong Yun-Zhan, 2014. A Hybrid Intelligent Search Algorithm for Automatic Test Data Generation. Hindawi Publishing Corporation Mathematical Problems in Engineering.
- [78] Ye J., Zhan Z, 2009. A software test cases automated generation algorithm based on immune principles. Springer-Verlag Berlin Heidelberg.